Frontend Requirements

Owner: Frontend track (1 member) · **Depends on:** `data-contracts.md`

The frontend is the face of the project. In reviews, examiners will mostly see this, so it needs to explain the project as well as display numbers. It has **two audiences**:

1. **Reviewers / visitors.** They want to see the forecast for SJCET, what the digitized history looks like, and whether transfer learning helped.
2. **The digitization member.** They need a fast tool to check and correct OCR'd cells.

---

1. Scope

#	Page	Audience	Priority
P1	**Review Tool**: correct OCR'd register cells	team	**Must, build first**
P2	**Forecast**: next-days forecast at SJCET + node map	reviewers	Must
P3	**History**: explore the digitized 20-year register	reviewers	Must
P4	**Results**: model vs baselines, the transfer-learning ablation	reviewers	Must
P5	**About**: project, pipeline diagram, team	reviewers	Should

Out of scope: user accounts, public deployment, mobile apps, real-time station ingestion.

> **Important: the forecast is "replay" by default.** ERA5 data reaches us with a ~5-day delay and the register is not live, so the main demo is picking a past date and showing the forecast *next to what actually happened*. A live mode can come later.

2. Tech stack (recommended)

Layer	Choice	Why
Backend	**FastAPI** (Python, in `frontend/api/`)	Same language as the model; can call `predict()` directly; serves CSV/JSON/images
Frontend	**React + Vite + TypeScript** (`frontend/web/`)	Standard, fast dev server
Charts	**Recharts** (or Plotly.js for zoomable time series)	
Map	**Leaflet** via `react-leaflet` + OpenStreetMap tiles	Free, no API key
Styling	Plain CSS with variables, or Tailwind	
Package mgmt	`uv` for Python, `npm` for web	Team already uses uv

If time gets tight, **Streamlit** can replace P2–P4 (not P1: the review tool needs keyboard-driven input that Streamlit handles badly).

---

3. Functional requirements

IDs are for tracking in reviews and tests.

P1: Review Tool

ID	Requirement
FR-R1	Load the queue from `cells.csv`. Default queue = cells with `qc_flag != ok` **or** `ocr_conf < 0.7`, minus cells already in `corrections.csv`.
FR-R2	Filters: by `field`, `qc_reason`, page, year, and "random sample of ok cells" (for spot checks).
FR-R3	For the current cell, show: (a) the **zoomed crop**, (b) the **full page** with the cell's `bbox` highlighted and auto-scrolled into view, (c) the **row context** (the other values on the same date), (d) OCR text, confidence and QC reason.
FR-R4	Input box pre-filled with `ocr_text`, focused automatically.
FR-R5	Actions: **Accept** (Enter on unchanged text), **Correct** (Enter after editing), **Illegible**, **Blank**, **Skip**, **Back**.
FR-R6	Every action appends one row to `corrections.csv` (via `POST /api/review/cell/{id}`) and advances to the next cell in < 300 ms.
FR-R7	Validate input live: must match `^\d{1,3}(\.\d{1,2})?$` (or a date for `field=date`). Warn, but don't block, if it is outside the field's plausible range (table in §3.6).
FR-R8	Progress bar: reviewed / total in the queue, plus cells per minute.
FR-R9	Reviewer initials asked once and stored in `localStorage`.
FR-R10	Undo the last action (Back) re-opens that cell and appends a new correction.

Keyboard map (shown in a `?` overlay):

Key	Action
`Enter`	accept / save correction
`Ctrl+I`	illegible
`Ctrl+B`	blank
`Ctrl+→` / `Ctrl+←`	skip / back
`Ctrl+Z`	zoom crop 2×

P2: Forecast

ID	Requirement
FR-F1	Date picker (default = last date with data). Calls `GET /api/forecast?date=`.
FR-F2	Show 1–3 **day cards**: date, Tmax, Tmin, rain amount, rain probability (a bar or %), and a weather icon derived from rain (`rain_prob ≥ 0.6 & rain_mm ≥ 2.5` → rain icon).
FR-F3	In replay mode, each card shows **Observed** values beneath the forecast with the error (e.g. `+0.5 °C`).
FR-F4	**Map** centred on SJCET (9.727, 76.727), zoom ~8, showing graph nodes from `graph.json` coloured by `kind`; SJCET highlighted. Toggle to show edges. Click a node to see its lat/lon/kind.

ID	Requirement
FR-F5	Small "last 14 days + next 3 days" line chart of Tmax/Tmin with rain bars; forecast part dashed.
FR-F6	Model selector (if more than one model is in `metrics.json`): e.g. "GNN (pretrained)" vs "GNN (local only)".

P3: History

ID	Requirement
FR-H1	Read `sjcet_daily.csv` via `GET /api/history?from=&to=&vars=`.
FR-H2	**Coverage calendar heatmap** (years × days, coloured by fraction of fields present). This honestly shows gaps such as weekends and missing months.
FR-H3	Time-series chart of any selected variables, zoomable, year-range slider.
FR-H4	**Monthly rainfall climatology** (mean per month, with min/max band). Annual total rainfall bar chart with trend line.
FR-H5	Tmax/Tmin annual means with trend line and slope label (°C/decade).
FR-H6	Points with `qc_flag = suspect` drawn hollow; tooltip shows `source_page` and a link opening the page image.
FR-H7	Download the filtered range as CSV.

P4: Results

ID	Requirement
FR-M1	Metrics table from `metrics.json`: rows = models, columns = RMSE/MAE/bias per target; best value per column bolded. Baselines grouped separately.
FR-M2	**The headline chart:** RMSE vs local data fraction (10/25/50/100%), one line per training strategy (from scratch, 1→3, 1→2→3), mean ± std over seeds.
FR-M3	Predicted vs observed scatter and a time series for the test period, with a target selector.
FR-M4	Per-season error bars.
FR-M5	A one-sentence plain-English takeaway above each chart, written by the model team in `metrics.json` (optional `notes` field) or hard-coded.

P5: About

FR-A1: project summary, the 3-stage pipeline diagram (register → digitize → pretrain → fine-tune → forecast), data sources with credits (ERA5/Copernicus, NOAA GSOD, SJCET), team and guide.

3.6 Plausible ranges (for validation hints and chart axes)

field	min	max
`rain_mm`	0	400
`dry_bulb_c`, `wet_bulb_c`	15	40
`tmin_c`	15	30
`tmax_c`	22	40

field	min	max
`rh_pct`	30	100

---

4. API (FastAPI)

Method	Path	Returns
GET	`/api/health`	`{ok: true}`
GET	`/api/forecast?date=YYYY-MM-DD&model=`	Forecast JSON (contracts §4)
GET	`/api/forecast/dates`	list of dates a forecast can be made for
GET	`/api/graph`	Graph JSON (§6)
GET	`/api/metrics`	Metrics JSON (§5)
GET	`/api/history?from=&to=&vars=`	rows of `sjcet_daily.csv` as JSON
GET	`/api/review/queue?field=&reason=&page=&sample=`	ordered list of `cell_id`s
GET	`/api/review/cell/{cell_id}`	the `cells.csv` row + row context
GET	`/api/review/cell/{cell_id}/crop` · `/page`	PNG images
POST	`/api/review/cell/{cell_id}`	body `{value, status, reviewer}` → appends to `corrections.csv`

Until the model and data exist, serve mock files from `frontend/api/mock/`. Generate them with a small script (random but plausible: Tmax ≈ 31 ± 2, Tmin ≈ 23 ± 1, rain heavy in Jun–Sep). The API swaps to real files when `data/sjcet/` and `model/outputs/` exist. **Do not wait for other tracks.**

The forecast endpoint should call `model/predict.py:predict(date, model_name) -> dict` (provided by the model track), falling back to precomputed files in `model/outputs/forecasts/`.

---

5. UI / UX requirements

5.1 Principles

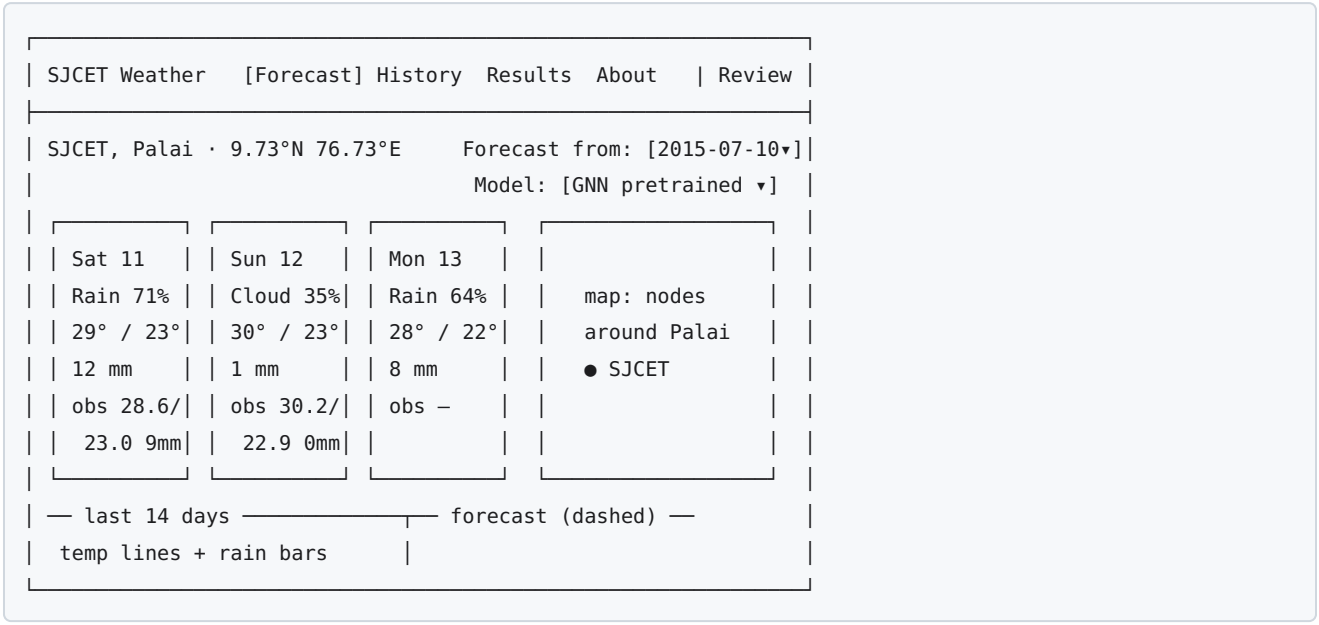
1. **Show the honest picture.** Missing data shows as gaps, not interpolated lines. Forecast vs observed are always visually distinct.
2. **One idea per screen.** Each page answers one question: *What will the weather be? What happened? Does pretraining help?*
3. **Review tool: speed over beauty.** The reviewer will check thousands of cells. Keep hands on the keyboard and never require a mouse.
4. **Readable for non-experts.** Units on every number, plain labels ("Max temp" rather than "tmax_c"), and tooltips that explain RMSE etc.

5.2 Layout & navigation

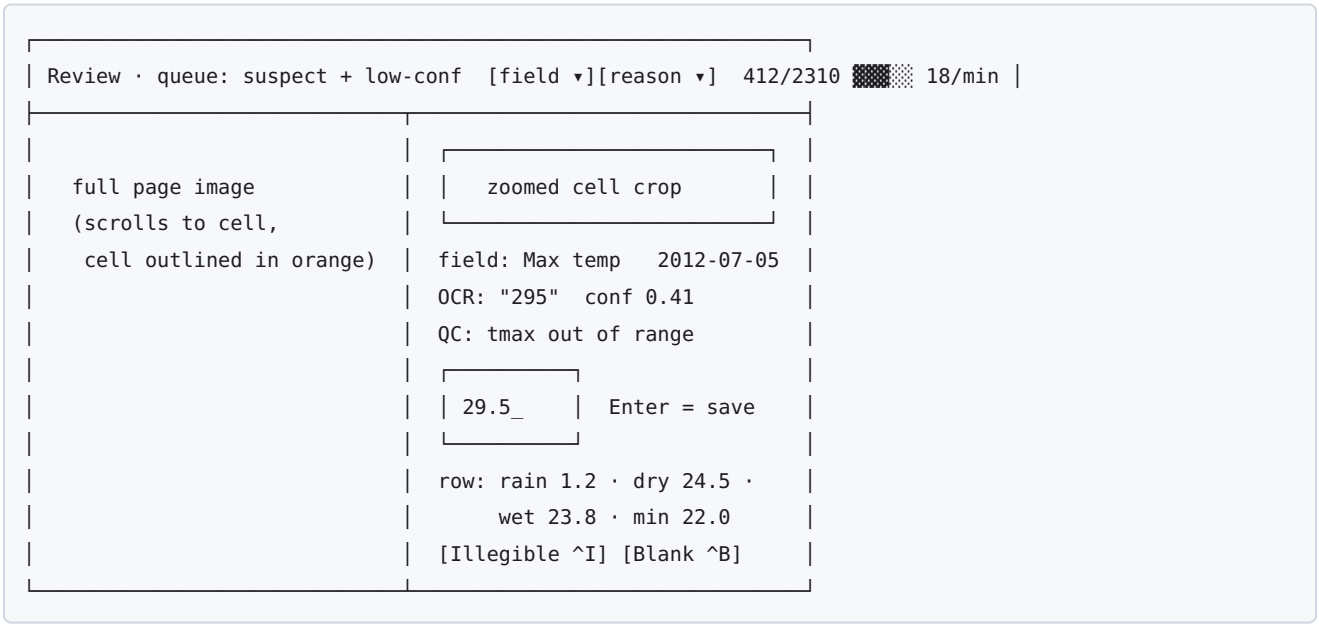
- Top bar: project name · tabs **Forecast | History | Results | Review | About**. Review can sit at the right, visually separated ("Team tools").
- Content max-width ~1200 px. Must work at 1366×768 (laptop projector) and down to ~375 px wide (phone) for P2–P5. The review tool is desktop only (≥ 1280 px).

5.3 Wireframes

Forecast



Review tool



5.4 Visual design

- **Colour tokens** (define as CSS variables, with light + dark themes): `--temp-max` warm red/orange, `--temp-min` blue, `--rain` teal/blue bars, `--observed` neutral dark, `--forecast` same hue as the variable but dashed/lighter, `--suspect` amber, `--error` red, `--ok` green.
- Use the same colour for the same variable on every page.
- Typography: one sans-serif family (e.g. Inter), tabular numerals (`font-variant-numeric: tabular-nums`) for all values.
- Map node colours: ERA5 grey dots (small), GSOD stations blue squares, SJCET a large accent marker with a label.
- Charts: no 3D, no pie charts. Label axes with units, start rain axes at 0.

5.5 States (every data view must handle all of these)

State	Behaviour
Loading	skeleton placeholders, not a blank page

State	Behaviour
Empty	clear message, e.g. "No register data for 2009 yet", and a hint
Error	message + retry button; the API error text in a collapsible detail
Partial	e.g. forecast exists but observed missing → show "–" with tooltip "not recorded"
Mock data	a visible **"Demo data"** badge whenever mock files are being served

5.6 Accessibility

- Colour contrast ≥ 4.5:1 for text. Never rely on colour alone: forecast vs observed also differ by line style; suspect points also differ by shape.
- Every interactive element is keyboard reachable with a visible focus ring.
- Charts have a text summary or "view as table" toggle.

5.7 Performance

- First load < 3 s on localhost. Review-tool next-cell < 300 ms. Preload the next 3 crops/pages in the background.
- History endpoint returns daily data (≈ 7,300 rows for 20 years), which is fine to send whole.

---

6. Folder layout

```
frontend/
├── api/                 FastAPI app (uv project)
│   ├── main.py
│   ├── routes/          forecast.py, history.py, review.py, results.py
│   └── mock/            generated mock JSON/CSV
└── web/                 Vite + React + TS
    └── src/
        ├── pages/       Forecast, History, Results, Review, About
        ├── components/  DayCard, NodeMap, TimeSeries, CoverageHeatmap, ...
        └── api.ts       typed fetch wrappers matching data-contracts.md
```

Run (target):

```
cd frontend/api && uv run fastapi dev main.py      # :8000
cd frontend/web && npm run dev                     # :5173, proxies /api → :8000
```

---

7. Milestones & acceptance

When	Deliverable	Done when
Week 1	API skeleton + mock data + **Review Tool**	Digitization member reviews 100 real cells with keyboard only; `corrections.csv` is written correctly
Week 2	Forecast + History pages on mock data	All 5 states visible; works at 1366×768 and 375 px
Week 3	Results page; switch to real `sjcet_daily.csv`	History shows real coverage heatmap
Later	Real model outputs plugged in	Replay forecast for a test-period date shows forecast vs observed

Test checklist before every review demo:

- [ ] Every page loads with the API stopped → error state, not a crash
- [ ] "Demo data" badge visible iff mocks are served
- [ ] Units on every number
- [ ] Keyboard-only run through the review tool